

Python-Implementable ML Concepts for Lab Exam

Key Python Libraries

- **scikit-learn (sklearn)**: The primary library for most ML algorithms, preprocessing, and metrics.
- **pandas**: For data loading, manipulation, and analysis (DataFrames).
- **NumPy**: For numerical operations and array manipulation.
- **SciPy (scipy.stats)**: For statistical tests and functions.
- **Matplotlib / Seaborn**: For data visualization and plotting.
- **Statsmodels**: For in-depth statistical modeling and tests (like F-test, J-test).

Supervised Learning Algorithms

- Bernoulli Naive Bayes (`sklearn.naive_bayes.BernoulliNB`)
- CART Algorithm (implemented as `sklearn.tree.DecisionTreeClassifier`)
- Decision Trees (`sklearn.tree.DecisionTreeClassifier`, `DecisionTreeRegressor`)
- Gaussian Naive Bayes (`sklearn.naive_bayes.GaussianNB`)
- Gradient Boosting (`sklearn.ensemble.GradientBoostingClassifier`, `GradientBoostingRegressor`)
- Gradient Descent (implemented as `sklearn.linear_model.SGDRegressor`, `SGDClassifier`)
- K-Nearest Neighbors (KNN) (`sklearn.neighbors.KNeighborsClassifier`, `KNeighborsRegressor`)
- Lasso Regression (L1) (`sklearn.linear_model.Lasso`)
- Linear Regression (`sklearn.linear_model.LinearRegression`)
- Logistic Regression (`sklearn.linear_model.LogisticRegression`)
- Multinomial Logistic Regression (set `multi_class='multinomial'` in `LogisticRegression`)
- Multinomial Naive Bayes (`sklearn.naive_bayes.MultinomialNB`)
- Multiple Linear Regression (use `LinearRegression` with multiple features)
- Naive Bayes (base concept, implemented as Gaussian, Multinomial, Bernoulli)
- Random Forests (`sklearn.ensemble.RandomForestClassifier`, `RandomForestRegressor`)
- Time-Series Models (e.g., ARIMA, via `statsmodels.tsa.arima.model.ARIMA`)
- Weighted Least Squares (supported in `statsmodels.api.WLS`)

clustering Unsupervised Learning Algorithms

- Anomaly Detection (e.g., `sklearn.ensemble.IsolationForest`, `sklearn.neighbors.LocalOutlierFactor`)
- Hierarchical Clustering (`sklearn.cluster.AgglomerativeClustering`)
- K-Means Clustering (`sklearn.cluster.KMeans`)
- Principal Component Analysis (PCA) (`sklearn.decomposition.PCA`)
- t-SNE (t-Distributed Stochastic Neighbor Embedding) (`sklearn.manifold.TSNE`)

Preprocessing & Feature Engineering

- Bag-of-words (BoW) (implemented by `sklearn.feature_extraction.text.CountVectorizer`)
- Binning (Discretization) (`sklearn.preprocessing.KBinsDiscretizer`, `pandas.cut`)
- Bootstrap Samples (Bootstrapping) (`sklearn.utils.resample`)
- CountVectorizer (`sklearn.feature_extraction.text.CountVectorizer`)
- Covariance Matrix (`numpy.cov`, `pandas.DataFrame.cov`)
- Data Visualization (Plotting) (`matplotlib.pyplot.hist`, `seaborn.scatterplot`, etc.)
- Date/Time Parts Extraction (using `pandas.to_datetime` and `.dt` accessor)
- Dimensionality Reduction (e.g., PCA, t-SNE)
- Encoding Categorical Variables (`sklearn.preprocessing.OneHotEncoder`, `LabelEncoder`)
- Euclidean Distance (`sklearn.metrics.pairwise.euclidean_distances`)
- Feature Bagging (Subsampling) (set `max_features` in Random Forest)
- Feature Selection (e.g., `sklearn.feature_selection.SelectKBest`, RFE)
- Feature Transformation (`sklearn.preprocessing.FunctionTransformer`)
- Handling Missing Values (Imputation) (`sklearn.impute.SimpleImputer`, `pandas.DataFrame.fillna`)
- Interaction Features (`sklearn.preprocessing.PolynomialFeatures`)
- Log Transformation (`numpy.log`, `sklearn.preprocessing.PowerTransformer`)
- ML Pipeline (creating a `sklearn.pipeline.Pipeline` object)
- Pearson Correlation (`pandas.DataFrame.corr`, `scipy.stats.pearsonr`)
- Polynomial Curve Fitting (use `PolynomialFeatures` + `LinearRegression`)
- Polynomial Features (`sklearn.preprocessing.PolynomialFeatures`)
- Random Samples with Replacement (`sklearn.utils.resample`)
- Scaling (Normalizing) Data (`sklearn.preprocessing.MinMaxScaler`, `StandardScaler`)
- Summary Statistics (`pandas.DataFrame.describe`)
- TF-IDF (`sklearn.feature_extraction.text.TfidfVectorizer`)
- Variance Thresholding (`sklearn.feature_selection.VarianceThreshold`)

Model Evaluation (Metrics)

- Accuracy (`sklearn.metrics.accuracy_score`)
- AIC / BIC (`statsmodels` models often return this, or `sklearn.metrics` for some models)
- Chi-square Test (`scipy.stats.chi2_contingency`)
- Confusion Matrix (`sklearn.metrics.confusion_matrix`, `ConfusionMatrixDisplay`)
- Elbow Method (plotting WCSS from `KMeans.inertia_`)
- F-Test (`scipy.stats.f_oneway`, `sklearn.feature_selection.f_regression`)
- F1 Score (`sklearn.metrics.f1_score`)
- Gini Impurity (criterion in `DecisionTreeClassifier`)
- Information Gain (Entropy) (criterion in `DecisionTreeClassifier`)
- Log-likelihood (`sklearn.metrics.log_loss`)
- Mean Absolute Error (MAE) (`sklearn.metrics.mean_absolute_error`)
- Mean Squared Error (MSE) (`sklearn.metrics.mean_squared_error`)
- P-value (from `scipy.stats` functions, `statsmodels` output)

- Precision (`sklearn.metrics.precision_score`)
- R-squared (R^2) (`sklearn.metrics.r2_score`)
- Recall (Sensitivity) (`sklearn.metrics.recall_score`)
- Residual Sum of Squares (RSS) (can be calculated from `mean_squared_error * n_samples`)
- Residuals (Prediction Errors) (calculated as `y_true - y_pred`)
- Root Mean Squared Error (RMSE) (`sklearn.metrics.mean_squared_error` with `squared=False`)
- Silhouette Method (`sklearn.metrics.silhouette_score`)
- Within-Cluster Sum of Squares (WCSS) (available as `KMeans.inertia_`)



Model Selection & Validation

- Backward Elimination (often implemented manually or via `sklearn.feature_selection.RFE`)
- Cross-Validation (CV) (`sklearn.model_selection.cross_val_score`)
- Forward Selection (part of `sklearn.feature_selection` or manual loops)
- Grid Search (`sklearn.model_selection.GridSearchCV`)
- Holdout Method (Train/Test Split) (`sklearn.model_selection.train_test_split`)
- K-Fold Cross-Validation (`sklearn.model_selection.KFold`, `cross_val_score`)
- Leave-One-Out Cross-Validation (LOOCV) (`sklearn.model_selection.LeaveOneOut`)
- Probability Threshold (manipulating `model.predict_proba()` output)
- Randomized Search (`sklearn.model_selection.RandomizedSearchCV`)
- Recursive Feature Elimination (RFE) (`sklearn.feature_selection.RFE`)
- Stratified K-Fold Cross-Validation (`sklearn.model_selection.StratifiedKFold`)